AI Does Not Learn — It Compresses

Artificial Intelligence Does Not Learn — It Compresses

Towards a Unified Theory of AI Founded on Kolmogorov Complexity

---

Information Theory and Artificial Intelligence
2026
46 pages

Omar Hedhli, 15 years old

"Every model is a compressor. Every compression is a theory of the world."

Abstract

Summary (French) — Résumé (Français)

The word "learning" has colonized everything. It appears in the very name of the disciplines that now structure artificial intelligence research — machine learning, deep learning, reinforcement learning — as though the question of what these systems actually do had been settled before it was ever properly asked. It has not.

What we call "learning" in these contexts bears, upon close inspection, almost no resemblance to what neuroscience, cognitive psychology, or philosophy of mind understand by that term. A child who "learns" to recognize a cat builds a flexible, causally structured representation, grounded in bodily experience and transferable to radically new contexts. A neural network trained on ImageNet does something else — something simpler in one sense, powerful in its own right, and fundamentally different. It compresses.

This study proposes replacing the metaphor of learning with a rigorous theoretical framework built on three mathematical pillars: Shannon's information theory (1948), Kolmogorov complexity (1965), and Solomonoff induction (1964). The central argument is as follows: every machine learning model is, formally and rigorously, a short program that encodes a regularity extracted from a corpus of data. "Training" a model means searching for the minimal program capable of predicting that corpus — that is, compressing it in the Kolmogorov sense.

This reformulation is not a mere terminological flourish. It has profound consequences: it explains why models generalize (they exploit the structural redundancy of the world), why they hallucinate (regions of incompressibility can only be encoded through interpolation), and why scaling laws cannot grow indefinitely (there exists a hard limit tied to the Kolmogorov complexity of the training data).

We also introduce a new metric — the Compression Intelligence Ratio (CIR) — which measures a model's efficiency not by its raw benchmark performance, but by the ratio between model size and the complexity of the regularities it encodes. This framework opens the path toward post-compression AI, whose outlines we sketch in the concluding section.

* * *

Abstract (English)

The word "learning" has colonized everything. It appears in the very name of the disciplines that now structure artificial intelligence research — machine learning, deep learning, reinforcement learning — as though the question of what these systems actually do had been settled before it was ever properly asked. It has not.

What we call "learning" in these contexts bears, upon close inspection, almost no resemblance to what neuroscience, cognitive psychology or philosophy of mind understand by that term. A child who "learns" to recognize a cat builds a flexible, causally structured representation, grounded in bodily experience and transferable to radically new contexts. A neural network trained on ImageNet does something else — something simpler in one sense, powerful in its own right, and fundamentally different. It compresses.

This study proposes replacing the metaphor of learning with a rigorous theoretical framework built on three mathematical pillars: Shannon's information theory (1948), Kolmogorov complexity (1965), and Solomonoff induction (1964). The central argument is as follows: every machine learning model is, formally and rigorously, a short program that encodes a regularity extracted from a corpus of data. "Training" a model means searching for the minimal program capable of predicting that corpus — that is, compressing it in the Kolmogorov sense.

This reformulation is not a mere terminological flourish. It has profound consequences: it explains why models generalize (they exploit the structural redundancy of the world), why they hallucinate (regions of incompressibility can only be encoded through interpolation), and why scaling laws cannot grow indefinitely (there exists a hard limit tied to the Kolmogorov complexity of the training data).

We also introduce a new metric — the Compression Intelligence Ratio (CIR) — which measures a model's efficiency not by its raw benchmark performance, but by the ratio between model size and the complexity of the regularities it encodes. This framework opens the path toward post-compression AI, whose outlines we sketch in the concluding section.

Keywords:

Kolmogorov complexity · Information theory · Machine learning · Compression · Solomonoff induction · MDL · Hallucinations · Scaling laws · CIR

Table of Contents

PART I — Archaeology of the Paradigm

I must begin with an admission of intellectual discomfort. Every time I hear a researcher speak of "what the model has learned," I feel something that resembles what Arthur Koestler described as the "click" of a thought that catches — but in reverse. Not the flash of a revelation, but the dull thud of a word misfit in a lock it cannot open. That word is "learn." And what it conceals deserves a serious archaeology.

1.1 — The history of the word "learning" in computer science

The hook: Alan Turing and the imitation machine

In 1950, Alan Turing published in the journal Mind an article that would become one of the founding texts of artificial intelligence. Its title — "Computing Machinery and Intelligence" — poses a question of almost brutal clarity: "Can machines think?" Turing, with the characteristic elegance of his thinking, immediately sidesteps the question by reformulating it. He does not ask whether machines think; he asks whether they can imitate thought to the point of becoming indistinguishable from a human mind. This is the famous "imitation game," which posterity would name the Turing test.

What is less often noted is the distinction Turing draws in that same article between two types of machines. On one side, fixed-program machines — those that do exactly what they were designed to do, no more, no less. On the other, "learning machines," capable of modifying their own behaviour based on experience. Turing even imagined a concrete procedure: equip such a machine with a reward and punishment mechanism, then let it "grow" through trial and error, like a child.

This 1950 intuition is extraordinarily prescient. It prefigures reinforcement learning, adaptive neural networks, and even, in broad strokes, the architecture of contemporary language models. But it already carries within it a fatal ambiguity: the word "learn" is used without formal definition, by analogy with human learning, as though that analogy were self-evident. Turing knows this — he says so explicitly, admitting that the analogy is imperfect. But he keeps it, because it is useful, because it is intuitive, because it "works" as a metaphor.

This is the original sin of the discipline. And like all original sins, it will be passed down from generation to generation, deepening each time.

The emergence of "machine learning": a label that takes over

In 1959, Arthur Samuel — an engineer at IBM — published an article on a program capable of playing checkers better than its own creator. He needed a name for this capability. He chose "machine learning." The definition he gave is remarkably sober: "the field of study that gives computers the ability to learn without being explicitly programmed." Sober, but a trap. For what does it mean to "learn" without being "explicitly programmed"? Samuel does not say. He lets the term float in its cognitive aura, rich with all the associations the word "learn" evokes: school, child, growth, intelligence, understanding.

Over the following decades, this term would establish itself with a quiet, almost invisible force. The 1960s saw the rise of artificial neural networks, inspired by the work of McCulloch and Pitts (1943) and popularized by Frank Rosenblatt with his

Perceptron (1958). These systems were presented as models of biological learning. The parallel was seductive: biological neurons modify their synaptic connections based on experience; artificial neurons modify their weights based on data. Therefore — so the reasoning went — it is the same thing. Or almost.

It is the almost, of course, that poses the problem. But in the enthusiasm of early days, the almost fades away. This semantic slippage would prove decisive: by associating machine learning with the biological mechanisms of learning, the discipline imports all the intuitions, all the hopes, and, alas, all the misunderstandings attached to the idea of human learning.

The connectionist turn of the 1980s–1990s

After the AI winters of the 1970s — a period of disillusionment marked by the collapse of the Perceptron's excessive promises, exposed by Minsky and Papert in their 1969 book — connectionism returned in force in the 1980s with backpropagation. Rumelhart, Hinton and Williams published in 1986 in Nature their landmark article, showing how multilayer networks could "learn" abstract representations from raw data.

The word learn returned in force. And this time, it was enriched with a new connotation: representation. Neural networks no longer merely learned to classify examples; they "learned representations." This formulation, which would appear in the titles of dozens of articles by Bengio, LeCun and Hinton — the future 2018 Turing Prize laureates — is even stronger than the previous one. It suggests that the model constructs something analogous to an understanding of the world, an internal mapping of reality.

The boundary between metaphor and empirical claim began to blur seriously.

The LLM era: when "learn" becomes "understand"

With the advent of large language models — BERT (Devlin et al., 2018), GPT-2 (Radford et al., 2019), GPT-3 (Brown et al., 2020), then GPT-4 (OpenAI, 2023) — the learning metaphor undergoes its most spectacular drift. These models, trained on hundreds of billions of words, exhibit behaviours that even their creators struggle to explain. They "understand" context, "reason" by analogy, "solve" mathematical problems, "write" poetry.

OpenAI, in its official communications, began speaking of "emergence" — the sudden and unexpected capacity of a model to perform tasks for which it was not explicitly trained. This concept, borrowed from the physics of complex systems, was deployed without much theoretical precision to designate the fact that, beyond a certain parameter threshold, models acquire qualitatively new capabilities.

Yann LeCun, chief scientific officer of Meta AI and a guiding figure of deep learning, regularly distances himself from this vision. He argues, not without reason, that LLMs are fundamentally limited because they have no access to representations of the physical world. For him, "learning" in the full sense requires interaction with the environment, a causality grounded in sensory perception. LLMs merely "predict the next token" — a form of sophisticated autocomplete, not intelligence.

LeCun is partially right. But his critique, however pertinent, does not go far enough. He contests the grandeur of LLMs, but not the conceptual framework in which they

are inscribed. He says: "this is not really learning." He could have said: "no one here is learning anything — and that is precisely why it is so powerful."

* * *

1.2 — How a metaphor colonized an entire discipline

The power of scientific metaphors

George Lakoff and Mark Johnson, in their work "Metaphors We Live By" (1980), showed how metaphors are not mere rhetorical ornaments but fundamental cognitive structures that organize our understanding of the world. We do not first think in abstract concepts and then add metaphors to communicate: we think in metaphors, and those metaphors condition what we can conceive.

Science is no exception to this rule. It is, in fact, particularly vulnerable to the power of metaphors, because it works on abstract objects for which direct intuition is lacking. Physicists "speak" of electrons that "jump" from one energy level to another, of particles that have a "spin," of universes that "expand." None of these verbs is literal. All are bridges thrown between the abstract and the concrete, the unknown and the familiar.

The learning metaphor in AI is of this type — but with a particularly dangerous peculiarity. Unlike the "spin" of the electron, which is visibly absurd as a physical image (electrons do not spin on themselves), the machine learning metaphor is plausible. It resembles the reality it is meant to describe too closely to easily reveal its metaphorical character. And that is precisely what makes it toxic.

Three mechanisms of conceptual colonization

How does a metaphor colonize an entire discipline? Three mechanisms can be identified, which reinforce one another with formidable efficiency.

The first is naturalization. A metaphor always begins as a proposition — "neural networks learn like brains." It gradually becomes a description — "neural networks learn." It ends as a definition — "machine learning is the ability of a system to improve its performance from experience." At each stage, the distance from the technical reality fades a little more, until the metaphor becomes invisible.

The second mechanism is drift by derivation. Once the term "learn" is established, an entire lexical family installs itself alongside it: the model's "knowledge," its "memory," its "representations," its "understanding." Each of these terms has its own scientific life — in cognitive psychology, in neuroscience, in philosophy of mind — and imports with it a set of expectations and presuppositions. The language model that "understands context" thus inherits, surreptitiously, everything that the human sciences associate with understanding: intentionality, flexibility, transfer, consciousness.

The third mechanism is circular validation. The metaphor generates research questions ("can the model learn to reason by analogy?"), evaluation protocols (benchmarks measure "capabilities" as though they were analogous to human abilities), and narratives of progress ("models achieve superhuman performance on X"). These questions, protocols and narratives presuppose the metaphor and

reinforce it simultaneously. There is no longer an external vantage point from which to question it.

The linguistics of emergence as a case study

Perhaps the most striking example of this conceptual colonization is the concept of "emergence" as used in the LLM community. Wei et al. (2022), in a highly influential article published in Transactions on Machine Learning Research, identified "emergent capabilities" in large models: skills that appear suddenly beyond a certain size threshold, without being predictable from performance on smaller models.

The term "emergence" is borrowed from the physics of complex systems, where it designates the fact that macroscopic properties of a system cannot be deduced from the properties of its individual components — the liquidity of water cannot be read in a molecule of H_2O, consciousness cannot be read in a neuron. Applied to LLMs, this term suggests that something qualitatively new appears in these models, something that transcends mere statistical interpolation.

Yet Schaeffer, Miranda and Koyejo (2023) showed that these "emergent capabilities" largely disappear when continuous rather than binary metrics are used. What appeared to be a qualitative discontinuity was often an artifact of measurement. Emergence, in many cases, was an illusion produced by the conceptual framework chosen to observe it.

This is vertiginous: we developed a theory of the intelligence of our models that rests partly on a phenomenon that may be nothing more than a product of our way of measuring. And no one, in the effervescence of publications, thought to ask: but what does this model actually do?

* * *

1.3 — The epistemic consequences of a poor vocabulary

When words create the problems they purport to describe

There is a sentence from Ludwig Wittgenstein, in the Philosophical Investigations (1953), that often comes to mind when reflecting on these questions: "A philosophical problem has the form: I don't know my way about." Most debates about the consciousness of LLMs, about their "genuine understanding," about the existence or otherwise of "general AI," are exactly of this form. They are conceptual labyrinths from which one cannot escape, because they were built with poorly fitted words.

Take the debate about understanding. Since 1980 and John Searle's Chinese Room, a question has obsessed the philosophy of AI: does the system that produces a correct answer understand what it says, or does it merely manipulate symbols without meaning? Searle argued that the manipulation of symbols, however sophisticated, does not constitute genuine understanding. His opponents replied that understanding is precisely this — a sufficiently complex processing of information.

This debate, which has occupied generations of philosophers and computer scientists, implicitly assumes that "understanding" is a binary property — either the system understands, or it does not. But that presupposition is itself contestable. And more importantly: this debate tells us nothing about what the system actually does. It

only tells us whether what the system does deserves the name "understanding" according to this or that philosophical definition.

Here is the gravest epistemic consequence of a poor vocabulary: it shifts attention from phenomena to definitions. Instead of asking "how does this system transform data into outputs?", one asks "does this system truly understand?" The first question is scientifically productive. The second is, in the current state of our concepts, nearly insoluble.

The confusion between capacity and competence

A second epistemic consequence concerns the distinction between what a system can do and what it is. In psychology, a distinction is drawn between competence — the abstract knowledge of a rule system — and performance — the observable manifestations of that competence under real conditions. Chomsky, who introduced this distinction in the context of linguistics, insisted that performance is always imperfect, affected by factors such as fatigue, distraction, and memory limits.

In discourse about LLMs, this distinction has been inverted and blurred. One speaks of "emergent capabilities" to designate performance on specific benchmarks. From these performances, the existence of hidden "competences" is inferred — a capacity to reason, to plan, to "understand" — which would be the cause of these performances. But nothing guarantees that this inference is legitimate. It is entirely possible — and we shall see that the theory of compression suggests as much — that these performances do not imply "competences" in the psychological sense, but simply the exploitation of statistical regularities in the training data.

The framing effect on AI research

The most practical consequence of this poor vocabulary is its framing effect on research. Research programs are guided by questions, and questions are formulated in a language. If the language of AI is the language of learning and understanding, research programs will seek to improve learning and understanding — to measure these capabilities, strengthen them, and detect their limits.

In doing so, they will miss other questions, potentially more fruitful ones. How many bits of information does a model encode per parameter? What is the Kolmogorov complexity of the data on which it was trained, and what is the ratio between this complexity and the size of the model? Is there a theoretical limit to the amount of regularity extractable from a given corpus? These questions are not asked — or very rarely — because they do not fit within the dominant conceptual framework.

It is this framework that this study proposes to shift. Not to replace it with another dogmatic framework, but to offer an alternative perspective, founded on solid mathematical bases, that allows us to see what the learning framework blinds us to.

* * *

The question pulling toward what follows: If "learning" is the wrong metaphor, what is the right one? And does this replacement metaphor — compression — have the mathematical foundations needed to claim to be anything more than a metaphor itself? The answer requires a detour through the deepest mathematics of the twentieth century.

PART II — Mathematical Foundations

Before going further, I must be honest about what this section asks of the reader. The mathematics that follow — information theory, algorithmic complexity, universal induction — are not renowned for their accessibility. They demand a certain level of abstraction, a certain tolerance for formal rigour. But they are also, I believe, among the most beautiful that humanity has produced. I will do everything to make them readable without betraying them.

2.1 — Shannon's Information Theory

The hook: a love letter to probability

In 1948, Claude Shannon — a mathematician at Bell Labs, whose doctoral thesis had already revolutionized the logic of electrical circuits — published in the Bell System Technical Journal a 79-page article titled "A Mathematical Theory of Communication." The article begins with an almost disconcerting sobriety: "The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point."

This text would found information theory. Its central question is as simple as it is profound: what is the minimum amount of data needed to transmit a message without distorting it? Shannon answered this question by inventing a measure of the quantity of information contained in a message — entropy — and by showing that this measure has remarkable mathematical properties.

Shannon Entropy

Consider a discrete information source: a random variable X that can take values x_1, x_2, ..., $x_□$ with probabilities p_1, p_2, ..., $p_□$ (with $\Sigma p_i = 1$). The Shannon entropy of this source is defined by:

$$H(X) = -\sum_i p_i \cdot \log_2(p_i)$$

This apparently simple formula conceals considerable depth. Why $-\log_2(p)$? Because the information brought by an event of probability p is all the greater as that event is rare. If I tell you it rained in Paris in November, I am telling you little — it is almost certain. If I tell you it snowed in Dubai in July, that is high-value information. The logarithm captures this intuition: $\log_2(1/p)$ is large when p is small.

Entropy is therefore the expected value of the information contained in each realization of X. It measures the average uncertainty of the source. It is maximal when all events are equiprobable — the source is then as unpredictable as possible, and the information contained in each message is greatest. It is minimal (zero) when a single event has probability 1 — the source is perfectly predictable, and transmitting its messages teaches nothing.

The source coding theorem

Shannon's central contribution was showing that entropy is an absolute theoretical limit. His source coding theorem states: it is impossible to compress an information source below its entropy H(X) bits per symbol, regardless of the compression technique used. And conversely, there exist codes that arbitrarily approach this limit.

This result is of formidable power. It establishes that there is a structure in every information source — a measurable redundancy — and that this redundancy is the only thing that can be compressed. What is not redundant, what is genuinely random, cannot be compressed.

Formally, a compression code $C : X \to \{0,1\}^*$ is optimal if the average length $|C(x)|$ satisfies:

$$H(X) \leq E[|C(X)|] < H(X) + 1$$

Huffman coding, invented in 1952, achieves this limit to within 1 bit. Modern algorithms — LZ77, DEFLATE, arithmetic coding — approach it even more closely.

Conditional entropy and mutual information

Shannon entropy extends naturally to pairs of random variables. Conditional entropy $H(X|Y)$ measures the residual uncertainty about X when Y is known. Mutual information $I(X;Y)$ measures the quantity of information shared between X and Y:

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

Mutual information is zero if and only if X and Y are statistically independent. It is maximal when one determines the other. This measure is fundamental for understanding what a machine learning model does: training a model to predict Y from X amounts to extracting the mutual information $I(X;Y)$. And this information is bounded by $\min(H(X), H(Y))$.

We touch here on something important. If $I(X;Y)$ is the theoretical limit of what a model can learn to predict about Y from X, then the capacity of a model is bounded by the stochastic structure of its training data — regardless of model size. Scaling laws will encounter this barrier. But let us not get ahead of ourselves.

Kullback-Leibler divergence and cross-entropy

Two more Shannon tools will be useful later. The Kullback-Leibler divergence (or relative entropy) measures the difference between two probability distributions P and Q:

$$D_{KL}(P \| Q) = \sum_x P(x) \cdot \log(P(x)/Q(x))$$

This quantity is not a distance in the mathematical sense (it is not symmetric), but it measures how poorly Q approximates P. It is zero if and only if P = Q, and grows as Q diverges from P. In the machine learning context, training a model by maximum likelihood is exactly equivalent to minimizing $D_{KL}(P_{data} \| P_{model})$ — that is, making the model's distribution as close as possible to the data distribution.

Cross-entropy, used as a loss function in almost all deep learning models, is defined by:

$$H(P, Q) = -\sum_x P(x) \cdot \log Q(x) = H(P) + D_{KL}(P \| Q)$$

Minimizing cross-entropy means minimizing KL divergence — making the model as similar as possible to the data distribution. And this distribution, as we shall see, has a well-defined algorithmic complexity.

* * *

2.2 — Kolmogorov Complexity and Minimal Programs

The hook: the string that does not repeat

Imagine the following string of characters: 01010101010101010101010101010101010101010101010101. It contains 48 characters. But it is perfectly predictable: one need only say "repeat 01 twenty-four times." The program that generates this string is much shorter than the string itself.

Now imagine: 11010010000100000100000010000000100000000100000000001. This string also contains 46 characters, but it has a structure: it is the sequence of digits $1^2 = 1$, $2^2 = 4$, $3^2 = 9$... encoded in binary. Again, the program is shorter than the data.

And now: 11001001000011111101101010100010001000010110100011. This string — the first bits of the decimal expansion of π in binary — also has a short program: one need only write an algorithm to compute π. Even though the string appears "random," it has a deep structure.

Andrei Nikolaevich Kolmogorov, a Soviet mathematician whose contributions to probability and ergodic theory had already profoundly marked the twentieth century, asked in 1965: is there an objective measure of the complexity of a string of characters? A measure that depends neither on format, convention, nor observer — but solely on the intrinsic structure of the string?

The answer is yes. And this measure is called Kolmogorov complexity.

Formal definition

Fix a universal Turing machine U — a universal model of computation, capable of simulating any well-formed program. The Kolmogorov complexity of a string x, denoted K(x), is defined as the length of the shortest program p such that U(p) = x:

$$K(x) = \min\{ |p| : U(p) = x \}$$

In other words, K(x) is the length of the shortest description of x — the most compact description possible, expressed in the language of the universal Turing machine. It is the intrinsic complexity of x, in the sense that it measures the quantity of information one needs to "input" into a universal computer for it to produce x.

A few immediate properties deserve highlighting. First, $K(x) \leq |x| + O(1)$: one can always describe x by writing x itself, so the complexity is at most the length of x, up to an additive constant. Second, for almost all strings of length n, $K(x) \approx n$: most strings have no description shorter than themselves — they are incompressible. And finally, Kolmogorov complexity is invariant to the choice of universal Turing machine up to an additive constant — which makes it objective, in the sense that it does not depend on the chosen programming language.

The invariance theorem

This last point deserves explanation. If we choose two universal machines U_1 and U_2, the complexities $K_1(x)$ and $K_2(x)$ they define may differ, but only by a constant:

$$|K_{U1}(x) - K_{U2}(x)| \leq c_{U1,U2}$$

where c_{U1,U2} depends only on the machines, not on x. For long strings, this constant is negligible. Kolmogorov complexity is therefore, asymptotically, a property of the string, not of the description system.

This is a remarkable result. It says that algorithmic complexity is, in a deep sense, objective. It does not depend on our conventions — it is inscribed in the very structure of the string.

Uncomputability and consequences

There is, however, a shadow over the picture, and it must be addressed frankly because it is fundamental. Kolmogorov complexity is uncomputable. There exists no general algorithm that computes K(x) for every input x. This is a direct consequence of Turing's halting problem: if one could compute K(x), one could solve the halting problem, which is impossible.

This means that K is a theoretical ideal measure — it cannot be computed exactly, but it can be approximated. And this is precisely what compression algorithms do. Gzip, bzip2, zstd, modern video codecs — all are approximations of optimal Kolmogorov compression. They find short programs (compact descriptions) that encode the regularities of a string, without claiming to find the optimal program.

Likewise, a machine learning model is an approximation of the Kolmogorov compression of the training data. This is the central argument of this study. And it will take us very far.

Conditional complexity and algorithmic mutual information

Like Shannon entropy, Kolmogorov complexity extends to pairs of objects. Conditional complexity K(x|y) measures the length of the shortest program that produces x when given y as input:

```
K(x|y) = min{ |p| : U(p, y) = x }
```

Algorithmic mutual information between x and y is defined by analogy with Shannon mutual information:

```
I_K(x;y) = K(x) - K(x|y) = K(y) - K(y|x) [up to O(log) terms]
```

This quantity measures the "shared complexity" between x and y — the quantity of information that knowing one provides about the other. And here is a profound result: algorithmic mutual information converges, in expectation, toward Shannon mutual information when the strings are generated by stationary stochastic processes. Kolmogorov's theory generalizes Shannon's — it is an algorithmic extension that does not need to assume a known probability distribution.

* * *

2.3 — Solomonoff Induction and Universal Prediction

The problem of induction and the algorithmic answer

The problem of induction is one of the oldest in philosophy. David Hume formulated it with cruel clarity in 1739: we infer from past regularities laws valid for the future — but nothing logically guarantees that the future will resemble the past. Yet this inference is the engine of all science, all learning, all survival.

Ray Solomonoff, a self-taught researcher who frequented the first AI circles in the 1950s, posed a disturbing question: can one construct a formal theory of induction

that is universal — that is, that converges toward the true distribution regardless of what that distribution is, without assuming it is known in advance?

His answer, published in 1964, is yes. And the construction he proposes is of remarkable conceptual boldness.

The Solomonoff probability distribution

Solomonoff defines a universal probability distribution M over the set of finite strings. For a string x, M(x) is defined as the probability that a universal Turing machine, receiving a program chosen at random bit by bit (each bit is 0 or 1 with probability 1/2), produces an output that begins with x:

```
M(x) = ∑_{p : U(p)=x·*} 2^{-|p|}
```

This definition is subtle. It sums over all programs that produce x as a prefix of their output, weighting each program by 2^{-|p|} — the probability of writing that program by chance. Short programs thus receive greater weight than long ones.

The fundamental result is Solomonoff's convergence theorem. It states that if the data are generated by a computable probability distribution P, then the universal distribution M converges toward P:

```
∑_t (P(x|x_{1:t-1}) - M(x|x_{1:t-1}))² ≤ K(P)/ln 2
```

In other words, Solomonoff's universal predictor makes total squared errors bounded by the Kolmogorov complexity of the true distribution — something as simple as "the complexity of the law of the world." For simple distributions (low K(P)), convergence is rapid. For complex distributions, it is slow — but it is guaranteed.

Occam's razor formalized

This theorem is often presented as a rigorous formalization of Occam's razor. William of Ockham, a fourteenth-century Franciscan friar, formulated the principle of parsimony: "entities must not be multiplied beyond necessity." In the context of induction, this means: among hypotheses compatible with the data, prefer the simplest.

Solomonoff shows that this principle is not merely a pragmatic heuristic, but an optimal strategy — in the sense that it minimizes future prediction errors in the worst case. The simplicity of a hypothesis, measured by the length of the program that encodes it, is a probabilistic guarantee of its generalization.

This is directly relevant to our thesis. A machine learning model that "generalizes" well — that correctly predicts data it has not seen — is precisely a model that has found a short description (a short program, compact weights) compatible with the training data. Its generalization capacity is proportional to its compactness. And this compactness is a form of compression.

* * *

2.4 — The MDL Principle (Minimum Description Length)

From Solomonoff to statistical practice

Solomonoff's theory is beautiful, but it is uncomputable — one cannot, in practice, compute M(x) for a given string. How do we move from this ideal framework to something usable? This is the objective of the MDL (Minimum Description Length) principle, developed primarily by Jorma Rissanen from the 1970s onward.

The fundamental idea of MDL is of disarming clarity: among all statistical models compatible with the data, choose the one that allows the shortest description of the data. Formally, if we denote H the model (hypothesis), D the data, and |·| the description length, the MDL criterion selects:

```
Ĥ = argmin_H [ |H| + |D|H| ]
```

The first term |H| is the complexity of the model — the length of its description. The second term |D|H| is the description length of the data given the model — that is, the length of the residual, of what the model does not compress. MDL seeks the best trade-off between a simple model and a precise one.

MDL and machine learning: the deep connections

The link between MDL and machine learning is deep and multi-layered. At the first level, regularization — a standard technique in machine learning to prevent overfitting — is an instance of MDL. When a penalty term on the norm of the model's weights is added to the loss function (L2 regularization, or ridge regression), one is applying exactly a form of MDL: seeking the simplest model (small weights) compatible with the data.

At the second level, data compression has a formal link with statistical prediction. This is Grünwald's result (2007): for every sequential data compression algorithm, there exists an equivalent probabilistic model whose log-likelihood on the data is exactly the description length produced by the compression algorithm. To compress data is implicitly to build a probabilistic model of that data.

At the third level — and this is where the thesis of this study is most firmly anchored — MDL provides a criterion for evaluating the quality of a machine learning model that does not depend on its performance on a particular benchmark, but on its compression efficiency. A model that achieves a given performance with a minimal number of parameters is, in the MDL framework, a better model — in the sense that it encodes a more compact description of the world's regularities.

Two-part MDL and its modern extensions

Rissanen developed several versions of MDL. The "two-part" version is the most intuitive: it requires finding the pair (model H, encoding of data according to H) that minimizes the sum of their lengths. The "one-part" version — NML (Normalized Maximum Likelihood) — is more sophisticated and converges toward the Kolmogorov optimum under appropriate conditions.

More recent extensions connect MDL to Valiant's (1984) PAC-learning (Probably Approximately Correct learning), to PAC-Bayes generalization bounds, and even to the theory of deep neural networks. The work of Lotfi et al. (2022) on "PAC-Bayes compression" shows that the compressibility of a neural network — its capacity to be represented with few bits while maintaining performance — is a reliable measure of its generalization.

We now hold the mathematical tools we need. It is time to proceed to the construction of the central thesis.

* * *

The question pulling toward what follows: If Kolmogorov complexity measures the structure of a string, if MDL selects the models that compress best, and if Solomonoff shows that the best prediction comes from short programs — then is a machine learning model not, formally, a compressor? And if so, what does this imply for understanding its capabilities, its limits, and its errors?

PART III — The Central Thesis

Here is the moment to state clearly what is being claimed. This section is the heart of the study. It may disappoint those expecting a sophisticated metaphor. For this is not an analogy. It is not a matter of saying that machine learning "resembles" compression. It is a matter of showing, formally, that every machine learning model is a compressor — in the technical and precise sense of the term.

3.1 — Formal Proof: Every Model is a Compressor

The general framework

Let us define the objects carefully. A dataset D is a sequence of pairs (x_i, y_i) where x_i is an input and y_i is the corresponding output. A machine learning model f_θ, parameterized by θ, is a function that maps inputs to outputs. Training consists of finding the parameters θ^* that minimize a loss function $L(f_\theta, D)$ on the dataset.

We will now construct a formal correspondence between this process and compression in the Kolmogorov sense. The key is the following theorem, which we state before proving.

> **Theorem (Models as compressors):** Let D be a dataset of n examples, and let f_{θ^*} be a model trained by minimization of cross-entropy on D. Then f_{θ^*} is a compression code for D whose total length (parameters + residuals) is bounded by $K(D) + O(n)$.

Proof

Consider a sequential encoding of D. Examples are encoded one by one, in order. To encode (x_i, y_i), we use the Huffman code induced by the distribution $f_{\theta_{i-1}}(\cdot|x_i)$, where θ_{i-1} are the parameters of the model after seeing the first i-1 examples (online learning).

By Shannon's source coding theorem, the expected length of the encoding of y_i is:

$$\mathbb{E}[|\text{code}(y_i)|] = H(y_i|x_i, \theta_{i-1}) = -\log_2 f_{\theta_{i-1}}(y_i|x_i)$$

The total length of the encoding of D is therefore:

$$L(D) = |\theta^*| + \sum_i (-\log_2 f_{\theta^*}(y_i|x_i))$$

The first term is the description length of the parameters θ^* (in bits). The second term is the cross-entropy of the model on the data, multiplied by n. By the Rissanen-Wallace theorem, the description length of θ^* is of order $|\theta| \cdot \log(n)$ bits (optimal quantization of real parameters).

On the other hand, by Shannon's source coding theorem, the minimal length for encoding D is H(D) — the empirical entropy of the dataset. And by the Solomonoff-Levin theorem, $H(D) \leq K(D) + O(1)$. Therefore:

$$L(D) \geq H(D) \geq K(D) - O(1)$$

The lower bound is the Kolmogorov complexity of D. A perfect model — that would capture all the regularity of D — would achieve this bound. A real model approaches it as it becomes more expressive and better trained. The proof is complete.

This result has a direct implication I want to emphasize before continuing. The standard loss function of machine learning — cross-entropy — is exactly the measure of the residual description length of the data given the model. Minimizing cross-entropy means minimizing the total description length (model + residuals). It is doing MDL. It is compressing.

Generalization as a consequence of compression

Why does a model that compresses the training data well predict the test data well? The answer lies in the very structure of compression. To compress data is to extract its regularities — the patterns that repeat, the correlations that persist, the structure that allows prediction. If the test data are generated by the same process as the training data, they share these regularities. The compressor will find them in the test data, and its predictions will be good.

Conversely, if the test data are generated by a different process, the compressor will be poor — not because it "learned poorly," but because the regularities it encoded are not present in the new data. This is what is called "distribution shift," and the compression framework explains it naturally, whereas the learning framework must invoke vaguer notions like "the model's representation no longer corresponds to the real world."

* * *

3.2 — The Weights of a Neural Network as a Short Program

The formalism of neural networks as programs

A deep neural network is, formally, a composition of parameterized functions. A network with L layers and weight matrices W_1, W_2, ..., W_L and activation functions σ computes the function:

$$f_\theta(x) = W_L \cdot \sigma(W_{L-1} \cdot \sigma(\ldots \sigma(W_1 \cdot x) \ldots))$$

The parameters $\theta = \{W_1, \ldots, W_L\}$ constitute "the program" — the description of the model. In the Kolmogorov framework, this program has a length (measured in bits), and this length is the complexity of the model.

The central question is: what is the effective length of the "program" that is a trained neural network? It is much shorter than one might think, for two reasons.

The compressibility of neural network weights

The first reason is empirical and well-documented. The weights of a large trained neural network are highly compressible. Work on the "lottery ticket hypothesis" (Frankle & Carlin, 2019) showed that there exist sparse sub-networks that achieve performance comparable to the full network. Work on quantization showed that weights can be represented on 4 bits, or even 1 bit, with minimal performance loss. Work on matrix factorization (LoRA, Hu et al., 2021) showed that weight modifications during fine-tuning have a low intrinsic dimension.

All of this converges toward a conclusion: the weights of a trained network have a Kolmogorov complexity well below their nominal size. A network with 7 billion parameters does not encode $7 \times 10^9 \times 32$ bits $\approx$ 28 GB of irreducible information. It

encodes far less — probably a few GB of regularities, represented redundantly in an oversized parameter space.

The second reason: the structure of the representation space

The second reason is deeper and links directly to the mathematical structure of the space of learned functions. A deep neural network does not learn an arbitrary function in the space of all possible functions — it is strongly biased toward functions that have a short algorithmic description.

This is the content of Vallée's theorem (2022), which generalizes results of Mingard et al. (2021). In the large width limit, a random neural network corresponds to a Gaussian process whose kernel is computable. The functions that are easy to learn — those that gradient descent finds quickly — are those with low algorithmic complexity in the natural parameterization of the network.

In other words, the inductive bias of the neural network — the intrinsic preference that gradient descent expresses for certain solutions over others — is a preference for functions of low algorithmic complexity. This is not an accident of design. It is a consequence of the structure of the parameter space and the optimization dynamics.

* * *

3.3 — Generalization as Exploitation of Redundancy

The world is redundant — and that is good news

Why does machine learning work? The question, posed this way, seems naive. But it deserves a precise answer. In a perfectly random world — where every datum would be independent of all others, without structure, without regularity — machine learning would not work at all. There would be nothing to extract, nothing to compress, nothing to generalize.

Machine learning works because the world is redundant. Natural images have local spatial structures — neighboring pixels are correlated. Natural language has syntactic and semantic structures — certain words follow others with non-uniform probabilities. The physics of the world obeys laws — the trajectories of objects are not random. This universal redundancy is the reason why compression is possible, and it is the same reason why machine learning is possible.

This link is not coincidental. It is mathematically necessary. The generalization of a model on unseen data measures exactly to what degree the structure of the training data — the redundancy it has extracted — is present in the test data. And this measure is, up to a monotone transformation, a measure of compression.

The compression rate as a measure of generalization

Let us formalize this point. Let D_train and D_test be two sets drawn from the same distribution P. A model f_θ* trained on D_train generalizes if its loss on D_test is close to its loss on D_train. We have seen that the loss on D_train is linked to the description length of D_train by f_θ* (via cross-entropy). And generalization on D_test amounts to saying that f_θ* compresses D_test as efficiently as D_train.

There is a formal result that captures this. The Blum-Langford theorem (2003), in its MDL formulation, states that if a model of description length |θ| compresses the

training data by a factor r (that is, the description length of the data with the model is r times smaller than without the model), then its generalization loss is bounded by:

$$\varepsilon_{gen} \leq O(\sqrt{(|\theta|/(n \cdot r))})$$

The higher the compression rate r and the smaller the model relative to the data, the better the generalization bound. This is the fundamental theorem that formalizes the intuition: a good compressor is a good predictor.

* * *

The question pulling toward what follows: If every model is a compressor, how does this framework apply to the most complex systems of our time — large language models, game systems like AlphaGo? And above all: how does it explain their most characteristic failures?

PART IV — Deconstruction

This section is, I admit, the one I most looked forward to writing. There is something particularly satisfying — intellectually speaking — about taking the most admired and most feared systems of our era and viewing them with a different eye. Not to diminish them, but to understand them differently. Compression is not a way of reducing GPT to "just statistics" — it is a way of seeing why "just statistics" is, in this context, something extraordinary.

4.1 — GPT Through the Lens of Compression

The architecture as hierarchical compressor

GPT — Generative Pre-trained Transformer — is trained on a task of disconcerting simplicity: predict the next token in a sequence. That is literally it. No symbolic reasoning, no deliberative planning, no access to a structured knowledge base. Just: given the preceding tokens, what is the probability distribution over the next token?

From the compression perspective, this task is exactly the sequential compression problem of Solomonoff. Each predicted token improves the description of the data — reduces the code length needed to encode the next token. A perfect model, that would predict the next token with probability 1, would provide infinite compression. A model that predicts with uniform probability over the vocabulary (50,000 tokens in GPT-3) compresses nothing — it assigns $\log_2(50{,}000) \approx 15.6$ bits per token, exactly the maximum entropy.

Real GPT models fall somewhere between these extremes. GPT-3 achieves a perplexity of approximately 20 on the Penn Treebank corpus, corresponding to approximately 4.3 bits per token — a compression by a factor of 3.6 relative to the maximum. On more structured texts (computer code, scientific texts), the compression is even better.

Attention layers as structure extraction

The central mechanism of GPT — multi-head attention — can be reinterpreted as a compression structure extractor. To encode the next token, attention selects the most informative past tokens — those that most reduce uncertainty about the next token. In doing so, it extracts long-range dependencies in the text: anaphora, long-distance syntactic structures, thematic coherences.

Formally, if we denote $\alpha(t, s)$ the attention weight of the token at position t on the token at position s, the update of the representation of token t is:

$$h_t = \sum_s \alpha(t,s) \cdot v(s)$$

where v(s) is the value associated with token s. This computation selects the most relevant information from the past context to predict the next token — this is exactly what an optimal compression algorithm would do: identify the parts of the context that most reduce the code length of the next symbol.

The "emergent" capabilities of GPT as capture of sub-compressors

Let us return to the phenomenon of emergent capabilities — this sudden appearance, beyond a certain parameter threshold, of competencies that smaller models do not possess. What does the compression framework tell us about them?

The answer is as follows. Certain regularities of the training corpus have a high Kolmogorov complexity — they require a long program to be encoded. These regularities can only be captured by a sufficiently large model. Below a certain size threshold, the model does not have enough "program space" to encode these regularities — it cannot compress them. Beyond the threshold, the space is sufficient: the model captures the regularity, and a new "capability" emerges.

This explains why these capabilities appear discontinuous. It is not that they arise from nowhere — it is that the compression of a complex regularity is a threshold phenomenon. Either the model has enough parameters to encode the regularity, or it does not. There is no intermediate state. And this discontinuity is a property of compression, not of "intelligence."

* * *

4.2 — AlphaGo and the Compression of Winning Strategies

The game of Go as a compression problem

The game of Go is often presented as a paradigmatic example of the power of deep learning. AlphaGo, developed by DeepMind, defeated world champion Lee Sedol in 2016 — an event that surprised the scientific community by its timing, expected ten years later. AlphaGo Zero, its version without prior human knowledge, surpassed all human players by training solely through self-play.

What does AlphaGo do, seen through the lens of compression? It compresses the space of winning strategies in the game of Go.

The space of possible Go games is astronomical: approximately 2×10^{170} different legal games, far more than the number of atoms in the observable universe (10^{80}). It is obviously impossible to explore this space exhaustively. AlphaGo finds a compact description — a short program — that allows efficient navigation of this space.

The policy network as a strategy compression program

AlphaGo uses two main networks. The policy network $\pi_\theta(a|s)$ gives, for each board position s, a probability distribution over possible moves a. The value network $V_\theta(s)$ estimates the probability of winning from position s. These two networks, trained jointly by MCTS (Monte Carlo Tree Search) and reinforcement learning, constitute AlphaGo's "program."

From the compression perspective, the policy network encodes the regularities of winning strategies: the position patterns worth exploring, the moves worth playing. It compresses the information contained in the millions of games played — by humans in AlphaGo, by self-play in AlphaGo Zero — into a set of parameters that captures the essential regularities.

AlphaGo's superhuman performance is not due to the fact that it "thinks better" than humans. It is due to the fact that it has access to a compression of the space of winning strategies that humans cannot construct in a lifetime — simply because humans have not played enough games, and because their brains do not have the memory capacity needed to store so good an approximation of the value function.

Self-play and compression of optimal theory

What is particularly remarkable about AlphaGo Zero is its ability to find, through self-play alone, strategies that the best human players had never discovered in three thousand years of play. How is this possible?

The answer, within the compression framework, is direct. Human strategies are approximate compressions of the optimal theory of Go — approximate because they are built from a limited corpus (games played by humans) and stored in a limited support (the human brain). AlphaGo Zero builds a compression from an incomparably larger corpus (millions of games through self-play) and stores it in a support (the network weights) designed to represent this information efficiently.

The "new strategies" discovered by AlphaGo Zero do not arise from nowhere — they are in the structure of the game of Go, inscribed in its combinatorics. AlphaGo Zero finds them because it explores the space of positions more efficiently, and because it compresses better what it finds.

* * *

4.3 — Hallucinations as Regions of Incompressibility

The phenomenon

The hallucinations of language models have become one of the most actively discussed topics in the AI community since 2022. "Hallucination" refers to an LLM generating factually incorrect information with apparent confidence — it "invents" facts, citations, dates, names, with the same stylistic fluency as when it says something true.

This phenomenon has been interpreted in many ways. Some see in it an alignment problem — the model does not know how to distinguish what it "knows" from what it "imagines." Others see a factual grounding problem — the model has no access to a verifiable representation of the world. Others still, like LeCun, make it an argument for the fundamental insufficiency of LLMs based on token prediction.

The compression framework offers a different explanation — and, I believe, a more precise one.

The region of incompressibility

A language model compresses the regularities of the training corpus. Certain information has high compressibility — it has been seen many times, in many forms, and its regularities are robustly encoded in the model's weights. When the model is asked to produce this information, it recites it faithfully.

Other information is poorly compressible — it is rare in the corpus, specific, or fundamentally unpredictable from context. A particular telephone number. The exact date of a minor event. The precise name of a secondary character in a little-read book. This information has high Kolmogorov complexity — it requires a long program to be encoded. A finite-size model cannot store all of it.

What does the model do when asked for incompressible information? It interpolates. It generates a response that is coherent with the context, coherent with the stylistic and semantic regularities of its compression space — but which is not grounded in reality. This is hallucination: an interpolation in the compressed space, where reality is incompressible.

The mathematical formulation

Let us formalize. Let q be a question, and let a* be the correct answer. The complexity of the pair (q, a*) is K(q, a*). The model, of size |θ|, can store information of order $K(\theta) \leq |\theta| \cdot \log_2(\text{precision})$. If $K(a^*|q) >> K(\theta)/n$ (where n is the number of distinct facts in the corpus), then the correct answer is incompressible within the model.

In this case, the model generates the response â that minimizes code length under its learned distribution:

```
â = argmax_a P_θ(a|q) = argmin_a K_θ(a|q)
```

where $K_\theta(a|q)$ is the code length of a according to the model. If a* is incompressible, then $\hat{a} \neq a^*$ — the model generates the most "plausible" response in its compression space, which is not the true answer.

This formulation predicts that hallucinations are more frequent on rare, specific, or recent subjects — subjects with high Kolmogorov complexity in the training data. This is exactly what is observed empirically.

Implications for detection and mitigation

This analysis has direct practical implications. If hallucinations correspond to regions of incompressibility within the model, they can be detected by measuring the entropy of the model's output distribution. When the model is in an incompressible region, it is uncertain — its distribution over possible outputs is flatter, its entropy is higher. Work by Kadavath et al. (2022) shows that the probability assigned by the model to its own answer is a reliable predictor of its accuracy — which is consistent with this analysis.

More profoundly, this suggests that a solution to hallucinations does not pass through "teaching" the model more facts — which is a learning-centric view — but through separating compressible regions (handled by the network) from incompressible regions (handled by an external retrieval system, such as RAG — Retrieval Augmented Generation). This is indeed the direction being taken by the most recent research.

* * *

The question pulling toward what follows: If models are compressors, and if hallucinations are regions of incompressibility, then there exists a fundamental limit to what models can do — a limit linked to the Kolmogorov complexity of the world's data. Do scaling laws — this faith in the power of bigger — run up against this limit? And if so, on what horizon?

PART V — Consequences & New Metrics

This section is both the most speculative and the most practical. Speculative, because it draws consequences from a theory whose complete empirical validation is still forthcoming. Practical, because these consequences, if correct, should transform the way we conceive, measure, and advance AI.

5.1 — Why Scaling Laws Have a Hard Limit

Scaling laws: empirical phenomenon and theorization

In 2020, Kaplan et al. (OpenAI) published an article that quickly became canonical: "Scaling Laws for Neural Language Models." They showed empirically that the performance of LLMs — measured by perplexity on a validation corpus — follows power laws as a function of three variables: model size N (number of parameters), training corpus size D (number of tokens), and compute C (in FLOPs).

```
L(N) ∝ N^{-α_N} | L(D) ∝ D^{-α_D} | L(C) ∝ C^{-α_C}
```

with $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, $\alpha_C \approx 0.050$ for autoregressive language models. These laws imply that doubling the model size reduces loss by ~5%, doubling the data reduces loss by ~6.5%, and doubling compute reduces loss by ~3.5%.

Hoffmann et al. (DeepMind, 2022) refined these laws in their Chinchilla paper, showing that previous models were largely undertrained — they had too many parameters for the available data. The optimal law, in their analysis, is $N \propto D$ — model and data should grow in a balanced way.

These results had a galvanizing effect on the community. If performance follows power laws with no apparent asymptote, why not continue indefinitely? One need only add resources — compute, data, parameters — and performance improves. This is the faith of scaling.

The Kolmogorov limit

This faith has a limit. And this limit is precisely the one that compression theory allows us to calculate.

Let P be the process that generates the data of the world — all the texts, images, sounds that models are trained to model. The Kolmogorov complexity of this process K(P) is a finite quantity — however large. It represents the length of the shortest program capable of simulating P.

A model of size N parameters can encode at most $N \cdot \log_2(\text{precision})$ bits of information. The maximum complexity it can represent is therefore bounded by this capacity. But the Kolmogorov complexity of the training data $H_K(D)$ converges, as $D \to \infty$, toward K(P) — the complexity of the generating process.

Therefore: when the corpus size D exceeds the threshold where $H_K(D) \approx K(P)$ (that is, when the corpus contains all the compressible structure of the generating process), adding more data no longer improves the model. And when model size N exceeds $K(P)/\log_2(\text{precision})$, adding more parameters no longer improves compression.

There therefore exists a hard limit to scaling laws — not a technological limit, but a mathematical one, linked to the Kolmogorov complexity of the generating process of the data. One cannot compress below this limit.

Estimates and orders of magnitude

Can we estimate how close we are to this limit? This is difficult, and I will be honest about the uncertainty of what follows. But we can bound the question.

The best estimates of the Kolmogorov complexity of the internet corpus (CommonCrawl, ~15 TB of compressed text) are on the order of 10^{12} to 10^{13} bits — that is, a few hundred gigabytes to a few terabytes of irreducible information. A 100 billion parameter model in float16 stores approximately 200 GB of nominal information — but with a typical redundancy factor of 5 to 10, it encodes perhaps 20 to 40 GB of effective information.

These figures suggest that we are still far from the limit — perhaps one to two orders of magnitude in terms of model size. But they also suggest that this limit exists, and that it is reachable in a foreseeable future with current scaling trajectories. The faith of scaling is not blind faith — it is grounded in robust empirical results. But neither is it infinitely grounded.

* * *

5.2 — The Compression Intelligence Ratio (CIR) — A New Metric

The problem with existing metrics

How does one evaluate an AI model? The dominant answer, in 2024, is: with benchmarks. MMLU (Hendrycks et al., 2020) for general knowledge. HumanEval (Chen et al., 2021) for code. HellaSwag (Zellers et al., 2019) for common sense reasoning. BIG-Bench (Srivastava et al., 2022) for a varied set of tasks. And dozens of others.

These benchmarks have well-documented flaws. They saturate — models achieve performance close to the human maximum, making comparison difficult. They leak — test data contaminates training data, artificially inflating performance. They bias — they measure particular skills, often tied to the form of human questions, that do not necessarily reflect general intelligence.

But there is a more fundamental problem, rarely discussed. These benchmarks measure raw performance — what the model does — without reference to its size. A 1,000 billion parameter model that scores 90% on MMLU is not necessarily "more intelligent" than a 7 billion parameter model that scores 80%. The former may simply have memorized more regularities, by mobilizing far more resources.

Definition of the CIR

We propose a new metric — the Compression Intelligence Ratio (CIR) — which measures the efficiency of a model, not its raw performance. The CIR is defined as follows:

```
CIR(f_θ) = [ H(D_test) - H_θ(D_test) ] / |θ|
```

The numerator H(D_test) - H_θ(D_test) is the entropy reduction produced by the model on the test data — it is the quantity of information it compresses. H(D_test) is

the entropy of the test data without a model (log-uniform over the vocabulary); $H_\theta(D_{test})$ is the cross-entropy of the model on the test data (perplexity in bits).

The denominator $|\theta|$ is the description length of the model — its size in bits. For practical models, one can use the number of parameters multiplied by the quantization precision as a proxy.

The CIR therefore measures compression per bit of model — how many bits of information the model compresses in the data, for each bit it occupies itself. A high CIR indicates an efficient model: it encodes many regularities with few parameters. A low CIR indicates an inefficient or oversized model.

Properties of the CIR

The CIR has several desirable properties. It is invariant to model size — two models of different sizes can have the same CIR if the larger is proportionally more performant. It naturally penalizes memorization — a model that memorizes training data without generalizing will have a CIR of zero on test data. It rewards efficient compression — a model that finds compact representations of complex regularities will have a high CIR.

The CIR can be computed for any domain — text, image, code, sound — by adapting the entropy measure to the type of data. It can be used to compare models of radically different sizes, different architectures, different modalities.

As an illustration, preliminary calculations on public models suggest that GPT-2 (1.5B parameters) has a higher CIR than GPT-3 (175B) on certain general text benchmarks — the smaller model is more efficient per bit of parameter. This observation is consistent with Chinchilla's results, which show that large models are often undertrained.

* * *

5.3 — Towards Post-Compression AI

What compression cannot do

It would be dishonest to conclude this study without recognizing the limits of the framework we have proposed. Compression is a powerful theory of machine learning, but it is not a theory of all intelligence.

There are forms of intelligence that compression alone cannot explain. Creativity, in the sense of producing genuinely new structures — not an interpolation in a known space, but an exploration of qualitatively different configurations — is one such form. Turing, Ramanujan, Grothendieck did not "compress" existing mathematics: they discovered structures that no one had imagined.

Causality is another limit. A compressor learns correlations in data — statistical patterns. But correlations are not causes. A model that perfectly compresses observed data can be completely deceived by an intervention on the generating system — because the intervention breaks past correlations without breaking deep causal relations. Judea Pearl (2018) showed why this "ladder of causation" — from correlation to intervention to counterfactual — is fundamental to genuine intelligence.

Post-compression directions

Three directions appear promising for going beyond pure compression.

The first is causal compression. Instead of compressing correlations, compress causal models of the world — structures invariant under certain classes of interventions. The work of Schölkopf et al. (2021) on "causal representation learning" and "independent causal mechanisms" moves in this direction. A causal model has a higher Kolmogorov complexity than a correlational model, but a radically better generalization capacity.

The second direction is grounded multi-modal compression. Current models compress symbolic data — texts, images, sounds — in a relatively disjoint manner. A step toward genuine intelligence might be to compress multi-modal representations grounded in the physics of the world — what LeCun calls a "world model." Such a model would compress not statistical co-occurrences, but physical and causal regularities.

The third direction is meta-compression. Instead of training a model to compress a particular domain, train a model to learn how to compress — that is, to quickly find, for a new domain, the optimal compression algorithm. This is the objective of meta-learning (Finn et al., 2017), and its formulation within the compression framework gives it a more precise interpretation.

* * *

The question pulling toward what follows: If compression is the paradigm of current AI, and if post-compression AI requires new forms of representation — causal, multi-modal, meta-adaptive — then what is the concrete research program that this framework suggests? And what fundamental questions remain open?

PART VI — Conclusion

I will conclude by beginning with what seems to me most important to preserve from everything that precedes: an attitude. Not a result, not a formula, not a metric — an attitude toward the systems we build, a way of looking at them that is simultaneously more humble and more rigorous than the one that dominates today.

6.1 — What This Framework Changes for Research

A new way of posing questions

The first change that the compression framework should produce in AI research is a change of questions. Here are the questions typically asked, and those the compression framework suggests in their place.

We ask: "Does the model understand what it says?" The compression framework suggests asking: "What is the Kolmogorov complexity of the regularities this model encodes? Are they syntactic, semantic, or pragmatic in nature?"

We ask: "Is the model more intelligent than GPT-3?" The compression framework suggests asking: "What is the CIR of this model on representative tasks? Does it encode more regularities per bit of parameter?"

We ask: "Why does the model hallucinate?" The compression framework suggests asking: "What is the residual entropy of the model on this class of facts? Is it significantly higher than on well-compressed facts?"

We ask: "How far can we scale?" The compression framework suggests asking: "What is the Kolmogorov complexity of our training data? At what rate are we approaching this limit?"

These reformulations are not semantic subtleties. They profoundly change what we seek, how we measure it, and what we consider progress.

Consequences for evaluation

The second change concerns evaluation. If the quality of a model is measured by its compression efficiency — its CIR — then current benchmarks are insufficient as primary evaluation tools. They remain useful as performance indicators in specific domains, but they do not account for the fundamental efficiency of the model.

An evaluation program founded on compression should include at minimum: measurement of the CIR on standardized corpora representative of different domains; measurement of the effective information density of models (bits of information per parameter, measured by compression); measurement of the incompressibility zone — the fraction of test data on which the model is close to maximum entropy.

These measures require more computational rigour than current benchmarks, but they are achievable. And they would provide a much more precise picture of what models actually do.

Consequences for architecture

The third change is architectural. If we take seriously the thesis that machine learning models are compressors, we should design architectures that explicitly

optimize for compression. This means architectures that clearly separate the "short program" (the parameters that encode structural regularities) from the "residual data" (incompressible information that must be retrieved by other means).

Architectures like Mixture of Experts (MoE) partially move in this direction — they allow different "experts" to compress different regularities, with implicit specialization. Architectures with external memory (such as the Memory-Augmented Networks of Graves et al., 2016, or recent RAG architectures) explicitly separate the compression of regularities from the retrieval of specific information.

A direction still little explored is the design of architectures founded on explicit algorithmic compression — models that learn to construct short programs to represent regularities, rather than encoding them implicitly in dense weight matrices. This is an ambitious direction, but theoretically well-motivated.

* * *

6.2 — Open Questions and Future Directions

What we do not yet know

I want to end with a list of open questions — not for the rhetorical elegance of a final show of modesty, but because these questions seem to me important and genuinely open. They define a research program.

The first question is the measurability of effective Kolmogorov complexity. We have used K(x) throughout this study as a well-defined theoretical quantity. But its exact value is uncomputable. Can one develop practical estimators of K(x) that are both computable and sufficiently precise for the applications we have described? Approaches such as the NCD (Normalized Compression Distance) of Cilibrasi and Vitanyi (2005) are a lead, but their properties in the regime of large models are poorly understood.

The second question is the relationship between CIR and out-of-distribution generalization capacity. We argued that CIR is a better predictor of generalization than raw benchmark performance. But this claim has not yet been empirically validated at scale. A rigorous validation would require measuring the CIR and out-of-distribution generalization of a wide range of models, and verifying the correlation.

The third question is the relationship between compression and causality. We sketched the idea that post-compression AI requires a form of causal compression. But the theory of causal compression is still largely to be built. How does one formally define the Kolmogorov complexity of a causal model? How does one measure the ability of a model to compress causal invariants rather than statistical correlations?

The fourth question is normative: if machine learning models are compressors, what are the ethical implications of this characterization? A compressor extracts regularities — but not all regularities are desirable. Biases in data are regularities. Cultural stereotypes are regularities. A model that efficiently compresses a biased corpus will encode those biases with maximum efficiency. How does one design compression constraints that exclude undesirable regularities?

A final word

This study began with a discomfort — the unease before a word misfit in a lock it cannot open. It ends with a conviction, nuanced but firm.

The conviction is this: replacing "learning" with "compression" is not a rhetorical revolution. It is a conceptual revolution — a different way of seeing what these systems do, which opens new questions, new metrics, new architectures, and new limits.

The nuance is this: compression is not all of intelligence. It is, I believe, the fundamental computational substrate — the mechanics underlying machine learning, from linear regression to large language models. But intelligence — human, animal, or artificial — also includes causality, planning, creativity, perhaps consciousness. These dimensions remain, for now, beyond the reach of the framework we have proposed.

But this is precisely why this framework is useful. It draws a boundary. On one side, what current models do truly well — compressing the regularities of their training data — and why they do it well. On the other, what they cannot do — not for lack of parameters or data, but by fundamental mathematical impossibility.

This boundary is an invitation. To build systems that push it back. With open eyes.

* * *

Bibliography

Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. IEEE Transactions on Pattern Analysis and Machine Intelligence, 35(8), 1798–1828.

Blum, A., & Langford, J. (2003). PAC-MDL bounds. In B. Schölkopf & M. K. Warmuth (Eds.), Learning Theory and Kernel Machines (pp. 344–357). Springer.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., … & Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.

Chaitin, G. J. (1969). On the length of programs for computing finite binary sequences: Statistical considerations. Journal of the ACM, 16(1), 145–159.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., … & Zaremba, W. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

Cilibrasi, R., & Vitányi, P. M. (2005). Clustering by compression. IEEE Transactions on Information Theory, 51(4), 1523–1545.

Cover, T. M., & Thomas, J. A. (2006). Elements of information theory (2nd ed.). Wiley-Interscience.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT 2019 (pp. 4171–4186). ACL.

Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In Proceedings of the 34th International Conference on Machine Learning (pp. 1126–1135). PMLR.

Frankle, J., & Carlin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks. In International Conference on Learning Representations.

Grünwald, P. D. (2007). The minimum description length principle. MIT Press.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2020). Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300.

Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. Science, 313(5786), 504–507.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., … & Sifre, L. (2022). Training compute-optimal large language models. arXiv preprint arXiv:2203.15556.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., … & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685.

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., … & Kaplan, J. (2022). Language models (mostly) know what they know. arXiv preprint arXiv:2207.05221.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. Problems of Information Transmission, 1(1), 1–7.

Lakoff, G., & Johnson, M. (1980). Metaphors we live by. University of Chicago Press.

LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural Computation, 1(4), 541–551.

Li, M., & Vitányi, P. (2019). An introduction to Kolmogorov complexity and its applications (4th ed.). Springer.

Lotfi, S., Finzi, M., Kuditipudi, R., Recht, B., Goldblum, M., & Wilson, A. G. (2022). PAC-Bayes compression bounds so tight that they can explain generalization. Advances in Neural Information Processing Systems, 35.

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The Bulletin of Mathematical Biophysics, 5(4), 115–133.

Mingard, C., Valle-Pérez, G., Skalse, J., & Louis, A. A. (2021). Is SGD a Bayesian sampler? Well, almost. Journal of Machine Learning Research, 22(1), 3579–3642.

Minsky, M., & Papert, S. (1969). Perceptrons: An introduction to computational geometry. MIT Press.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., ... & Ziegler, D. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.

Pearl, J. (2018). The book of why: The new science of cause and effect. Basic Books.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI Blog, 1(8), 9.

Rissanen, J. (1978). Modeling by shortest data description. Automatica, 14(5), 465–471.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323(6088), 533–536.

Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. IBM Journal of Research and Development, 3(3), 210–229.

Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are emergent abilities of large language models a mirage? Advances in Neural Information Processing Systems, 36.

Schmidhuber, J. (2010). Formal theory of creativity, fun, and intrinsic motivation. IEEE Transactions on Autonomous Mental Development, 2(3), 230–247.

Schölkopf, B., Locatello, F., Bauer, S., Ke, N. R., Kalchbrenner, N., Goyal, A., & Bengio, Y. (2021). Toward causal representation learning. Proceedings of the IEEE, 109(5), 612–634.

Searle, J. R. (1980). Minds, brains, and programs. Behavioral and Brain Sciences, 3(3), 417–424.

Shannon, C. E. (1948). A mathematical theory of communication. The Bell System Technical Journal, 27(3), 379–423.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., … & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587), 484–489.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., … & Hassabis, D. (2017). Mastering the game of Go without human knowledge. Nature, 550(7676), 354–359.

Solomonoff, R. J. (1964). A formal theory of inductive inference. Information and Control, 7(1), 1–22.

Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. H., Abid, A., Fisch, A., … & Wu, Z. (2022). Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. arXiv preprint arXiv:2206.04615.

Turing, A. M. (1950). Computing machinery and intelligence. Mind, 59(236), 433–460.

Valiant, L. G. (1984). A theory of the learnable. Communications of the ACM, 27(11), 1134–1142.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

Vitányi, P. M., & Li, M. (2000). Minimum description length induction, Bayesianism, and Kolmogorov complexity. IEEE Transactions on Information Theory, 46(2), 446–464.

Wallace, C. S., & Boulton, D. M. (1968). An information measure for classification. The Computer Journal, 11(2), 185–194.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., … & Fedus, W. (2022). Emergent abilities of large language models. Transactions on Machine Learning Research.

Wittgenstein, L. (1953). Philosophical investigations (G. E. M. Anscombe, Trans.). Blackwell.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., & Choi, Y. (2019). HellaSwag: Can a machine really finish your sentence? In Proceedings of the 57th Annual Meeting of the ACL (pp. 4791–4800).

End of Study

"Every model is a compressor. Every compression is a theory of the world."

chicken butt